Pack 04 · Measure and operate

Serving metrics, load testing, percentiles, SLOs and capacity planning. Lessons 01, 06, 08, 15, 16, 20–25, 27, 30, 43. Answer each question out loud in about a minute, then check the card.

Q1 · "Request X is 4× slower" PRIORITY

Lessons: 08, 16, 20

> **INTERVIEWER**
>
> Request X took 8.3 s end to end with 400 output tokens; request Y took 2.0 s with 85. Both show TTFT 300 ms and TPOT 20 ms. The PM says X is 4× slower. Is it, and what should the PM compare?

60-second answer

- **The latency formula:** end-to-end time is the wait for the first token plus one TPOT gap for every token after it, so E2E ≈ TTFT + TPOT × (N − 1). For X that's 0.3 + 399 × 0.02 ≈ 8.3 s, and for Y it's 0.3 + 84 × 0.02 ≈ 2.0 s, which matches both measurements.
- **Why X isn't slower:** both requests have the same TTFT and the same TPOT, so the server is serving them at **identical speed**. X takes longer only because it wrote more tokens (400 versus 85).
- **What the PM should compare:** **TTFT**, which is how long the user waits for the first token, and **TPOT**, which is how fast the stream runs after that. E2E can't be compared across requests of different lengths, because a longer answer takes longer even at the same speed.
- **Getting TPOT back from E2E:** after the first token there are N − 1 gaps, so TPOT = (E2E − TTFT) ÷ (N − 1). That's why you divide by N − 1, not N.

Key numbers: TPOT = (E2E − TTFT) ÷ (N − 1); e.g. (2.3 − 0.3) s ÷ 400 = 5 ms for 401 tokens.

Likely follow-up: "Average TPOT looks fine but users say the stream stutters?" The mean hides uneven gaps, so I'd look at the **distribution of inter-token gaps** (p95, p99, max) instead. The usual cause is long prefills of other requests, which interrupt the stream.

> **TRAP**
>
> I divided by N instead of N − 1, and blamed stutter on queue length (that explains slow starts, not uneven gaps).

Q2 · Two tools, two TPOTs PRIORITY

Lessons: 20

> **INTERVIEWER**
>
> A load test ran 50 short chats (20 tokens, 50 ms gaps) and 10 long answers (1,000 tokens, 10 ms gaps). The dashboard says TPOT ≈ 13.5 ms; a teammate's script says ≈ 43 ms. Which is right for "what a typical user feels"? And after turning on speculative decoding, a client script reports 45 ms per message with 3 tokens per message.

60-second answer

- **Per request (one vote per request):** take each request's gap and average over the 60 requests: (50 × 50 + 10 × 10) ÷ 60 = 2,600 ÷ 60 ≈ **43 ms**. This is the teammate's number.
- **Per token (every gap counts):** a 20-token chat has 19 gaps and a 1,000-token answer has 999, so the average is (50 × 19 × 50 + 10 × 999 × 10) ÷ (950 + 9,990) = 147,400 ÷ 10,940 ≈ **13.5 ms**. This is the dashboard's number, and it's dominated by the 10 long answers because they supply 9,990 of the 10,940 gaps.
- **Which is right:** "typical user" means **per request**, because 50 of the 60 users felt 50 ms gaps. The per-token number mostly describes the 10 long answers.
- **The speculative decoding script:** speculative decoding sends tokens in chunks, so one message carries several tokens. The script timed messages, not tokens, so the real figure is 45 ms ÷ 3 = **15 ms per token**.

Key numbers: per-token TPOT weights long requests by their gap count.

Likely follow-up: "Which would you put in an SLO?" Per-request percentiles, such as p99 per-request TPOT, because that way every user counts once no matter how long their answer is.

> **TRAP**
>
> I mixed gap counts into the per-request vote and forgot to multiply by the number of requests in the per-token sum.

Q3 · Prefill speed from a TTFT sweep PRIORITY

Lessons: 15, 24

> **INTERVIEWER**
>
> Single-request TTFT, max_tokens = 1, medians: 256 tokens → 23 ms, 8,000 tokens → 280 ms. Model TTFT = fixed cost + prompt ÷ throughput. What are the prefill throughput and fixed cost, and what's in the fixed cost?

60-second answer

- **Throughput is the slope:** going from 256 to 8,000 tokens adds 7,744 tokens and 280 − 23 = 257 ms, so prefill runs at 7,744 ÷ 0.257 s ≈ **30k tokens/s**. Taking the difference between two points cancels the fixed cost.

- **Fixed cost is the intercept:** at 30k tokens/s the 256-token prompt itself takes 256 ÷ 30k ≈ 8.5 ms, so the rest of its 23 ms is overhead: 23 − 8.5 ≈ **14.5 ms**.
- **What's in the fixed cost:** everything that doesn't grow with prompt length, which is HTTP and tokenization, scheduling, the first decode step, and detokenizing and streaming the first token.

Key numbers: throughput = Δtokens ÷ ΔTTFT.

Likely follow-up: "Why not divide 256 by 23 ms?" Because the 23 ms includes the 14.5 ms fixed cost, so 256 ÷ 23 ms ≈ 11k tokens/s folds the overhead into the rate. That's almost 3× too low compared with the real 30k.

> **TRAP**
> I divided one point. Use two points: slope for the rate, intercept for the overhead.

Q4 · "The benchmark said we're fine" PRIORITY

Lessons: 21, 25

> **INTERVIEWER**
> A closed-loop benchmark with 32 concurrent users measured 4 s mean time in system. At launch, real users arrive at 10 req/s. What happens? Which load test should you have run, and why can a waiting load generator hide a 3 s stall?

60-second answer

- **What happens at launch:** by **Little's law**, throughput = requests in flight ÷ time in system, so the server finishes 32 ÷ 4 s = **8 req/s**. Real users arrive at 10 req/s, so the queue grows by 10 − 8 = 2 req/s, which is **120 requests waiting after a minute**, and it keeps growing without bound.
- **Why the benchmark looked fine:** a **closed loop** only sends a new request when an old one finishes, so when the server slows down, its arrivals slow down too. That's why it can never show overload.
- **Which tests to run:** to find max throughput, run a **closed-loop concurrency sweep**. To see latency under launch traffic, run an **open-loop test with Poisson arrivals at 10 req/s**, since real users don't wait for each other.
- **How a waiting client hides a stall (coordinated omission):** take a test scheduled at 5 req/s for 60 s, which is 300 requests. A client that waits for each response sends only ≈ 1 request during a 3 s stall instead of the 15 its schedule called for, so p99 never sees the stall. A client that sends on schedule records all 15 slow requests out of 300, which is 5% and well over 1%, so p99 lands inside the stall at ≈ 2.6 s.

Key numbers: L = λ × W; queue growth = arrivals − completions.

Likely follow-up: "Traffic grows 30% on a server at 70% busy?" Utilization becomes ρ = 0.7 × 1.3 = 0.91. Time in system scales with 1 ÷ (1 − ρ), which goes from 1 ÷ 0.3 ≈ 3.3 to 1 ÷ 0.09 ≈ 11.1, so requests take 11.1 ÷ 3.3 ≈ **3.3× longer**. That's why near saturation a small increase in traffic blows up latency.

> **TRAP**
>
> I explained why the test missed it but not what happens (compute the growth). For max throughput, sweep concurrency, not request rate.

Q5 · Percentile traps PRIORITY

Lessons: 06, 22

> **INTERVIEWER**
>
> Three dashboards: P99 TTFT doubled from 0.8 to 1.5 s with no deploy and unchanged per-request timings; ops averages four replicas' P99s (0.6, 0.7, 0.6, 3.0 s) into "fleet P99 = 1.2 s"; and an agent makes 20 sequential calls against an API with P99 = 2 s. What's wrong in each?

60-second answer

- **First dashboard, mix shift:** if more long prompts arrive, the blended P99 TTFT rises even though no request got slower, since prompt length drives TTFT. To check, segment by prompt length and compare each segment's latency and its share of traffic.
- **Second dashboard, percentiles don't average:** you have to merge the replicas' histograms first and then take the percentile. With four replicas, the slowest 1% of the fleet is ≈ 4% of the bad replica's requests, which is its **P96**. That's likely ≈ 2 s, so the true fleet P99 is **well above 1.2 s**.
- **Third dashboard, tails compound:** each call has a 1% chance of being slow, so the chance that at least one of the 20 calls is slow is $1 - 0.99^{20} \approx$ **18%** of tasks. For only 1% of tasks to be slow, each call needs to meet about **P99.95**, since $20 \times 0.05\% \approx 1\%$.

Key numbers: $1 - (1 - q)^k \approx k \times q$ for small q.

Likely follow-up: "How many samples does a P99 need?" You want about 100 samples beyond the percentile for ≈ 10% precision. Only 1% of samples land beyond P99, so that means about $100 \div 0.01 \approx$ 10,000 samples.

> **TRAP**
>
> I said "load increased" (that would show as queueing) and blamed response length for TTFT (prompt length drives TTFT). For the agent I read "1%" as per task.

Q6 · Is a 3.5% speedup real? PRIORITY

Lessons: 06

> **INTERVIEWER**
>
> Before: 5 runs averaging 20.0 ms TPOT. After a tweak: 5 runs averaging 19.3 ms. Each run wobbles by σ ≈ 0.8 ms. A teammate says "3.5% faster, ship it."

60-second answer

- **Noise on one average:** a single run wobbles by σ = 0.8 ms, and averaging n runs shrinks that to **σ ÷ √n**, so each 5-run average wobbles by 0.8 ÷ √5 ≈ 0.36 ms.
- **Noise on the difference:** comparing two averages combines both of their noises, which makes it **√2** larger: 0.36 × √2 ≈ 0.51 ms. The 95% band is about 2× that, so ≈ **±1.0 ms**.
- **The verdict:** the measured gap is 20.0 − 19.3 = 0.7 ms, which sits inside the ±1.0 ms noise band, so the speedup is **not proven**.
- **How to prove it:** the band 2 × 0.8 × √2 ÷ √n has to shrink to 0.7 ms or less, which needs **n ≈ 11+ runs each**. The other option is to pair the runs so the noise they share cancels.

Key numbers: 95% band on a difference ≈ ±2 × √2 × σ ÷ √n.

Likely follow-up: "Same idea for model quality?" Yes. Compare the two models question by question (a paired comparison), as in the quantization pack, so the noise they share cancels just like it does with paired runs.

> **TRAP**
>
> I had no mental model to start from: σ, then σ ÷ √n, then × √2 for a difference, then a gap beyond ≈ 2× that.

Q7 · Goodput and the operating point PRIORITY

Lessons: 23, 25, 27, 43

> **INTERVIEWER**
>
> SLO: TTFT < 500 ms and TPOT < 30 ms. At 6 req/s, 5% fail with 429; of the successes 90% meet TTFT and 85% meet TPOT. What's the goodput? And a sweep peaks at 5,000 tokens/s, but every user must get at least 100 tokens/s at P99. What do you put in the capacity doc?

60-second answer

- **Goodput:** only requests that are accepted and meet both SLOs count, so goodput = 6 × 0.95 × 0.90 × 0.85 ≈ **4.4 req/s**. That's only **73%** of the 6 req/s the dashboard shows.
- **Turning the user floor into a latency limit:** 100 tokens/s per user means each token can take at most 1 ÷ 100 s = **10 ms**, so P99 TPOT must stay ≤ 10 ms. In the sweep, pick the **highest concurrency that still passes** that limit (32 users, ≈ 3,400 tokens/s), never the 5,000 tokens/s peak, because the peak fails the SLO.
- **What goes in the capacity doc:** capacity = **min(saturation wall, SLO wall, memory wall)**, because whichever wall you hit first is the real limit.
- **Why the fastest config can lose:** big steps break TPOT first, and a low admission cap breaks TTFT first, so raw speed doesn't decide it. Pick the config by **SLO-safe** throughput.

Key numbers: goodput = rate × success rate × product of SLO pass rates.

Likely follow-up: "What's the memory wall?" It's the KV pool's token capacity divided by the tokens each request holds by the time it finishes, which gives how many requests fit at once. For long agent outputs it's often the binding wall, because each request ends up holding many tokens.

> **TRAP**
>
> I picked the point "closest to" the target and leaned toward the peak. "At least" means the rightmost point that still passes.

Q8 · "40 ÷ 8 = 5 GPUs" PRIORITY

Lessons: 43

> **INTERVIEWER**
>
> The busiest minute averages 40 req/s and one H100 holds the SLO up to 8 req/s. The manager says 5 GPUs. With a 20% burst margin and surviving one replica failure, how many? And when traffic exceeds plan, should you raise max_num_seqs from the tested 128 to 512?

60-second answer

- **Usable capacity per GPU:** a 20% burst margin means running each GPU at 80% of its SLO limit, so each one handles $8 \times 0.8 = 6.4$ req/s.
- **GPUs for the load:** $40 \div 6.4 = 6.25$, and you can't run a quarter of a GPU, so **7** GPUs serve the busiest minute.
- **Surviving a failure:** if one replica dies, the remaining $N - 1$ must still be at least 7, so $N - 1 \geq 7$ gives **8 GPUs**, not the manager's 5.
- **Don't raise max_num_seqs to 512:** packing in more sequences than the tested 128 makes users who are already streaming get slower, and **preemptions** start.
- **What to do instead:** protect the tested operating point. Keep the cap at 128, **time out or reject** the excess queue with 429s, and **autoscale early** so new capacity arrives before the queue builds.

Key numbers: $N - 1 \geq \text{peak} \div (\text{per-GPU SLO capacity} \times (1 - \text{margin}))$.

Likely follow-up: "Why not VTC for overload?" VTC balances service between tenants, but it doesn't shed load, so the server stays overloaded.

> **TRAP**
>
> I answered "N + 2" from memory instead of working the inequality.

Q9 · Should someone be paged? PRIORITY

Lessons: 23

> **INTERVIEWER**
>
> SLO: 99.5% good over 30 days, about 2 million requests a month. Over the last hour 3% of requests were bad. How big is the error budget, what's the burn rate, and how long until it's gone?

60-second answer

- **Error budget:** the SLO allows 100% − 99.5% = 0.5% of requests to be bad, and 0.5% × 2 M = **10,000 bad requests** a month.
- **Burn rate:** the last hour ran 3% bad against an allowed 0.5%, so the budget is burning 3% ÷ 0.5% = **6×** faster than planned.
- **Time left:** at 1× the budget lasts exactly 30 days, so at 6× it lasts 30 ÷ 6 = **5 days**. That's a fast burn, so yes, page someone.

Key numbers: days left = 30 ÷ burn rate.

Likely follow-up: "How do you avoid paging on blips?" Alert on burn rate over two windows, such as 1 h and 5 min, and page only when both are high. A short blip spikes the 5-minute window but not the hour, so it doesn't page.

> **TRAP**
>
> I divided 6 by 30. Sanity check: at 1× the budget lasts exactly 30 days, so faster burns leave fewer days.

Q10 · Autoscaling on the wrong signal PRIORITY

Lessons: 01, 30

> **INTERVIEWER**
>
> An autoscaler adds a replica when CPU passes 70%. During a spike, users wait 5+ s for the first token but CPU shows 12%, so nothing scales. What should it watch, and how long does a new replica take with 16 GB of weights at 2 GB/s?

60-second answer

- **Why CPU is the wrong signal:** in GPU serving the CPU sits mostly idle, so it stays at 12% even while users wait 5 s. **GPU memory** doesn't work either, because the KV pool is preallocated, so it always reads ≈ 90%.
- **What to watch:** scale on **engine signals**, which are requests waiting (queue growth), KV cache usage, and TTFT. These move when users actually start waiting.
- **Cold start:** a new replica has to load 16 GB of weights at 2 GB/s, so it takes at least 16 GB ÷ 2 GB/s = **8 s**, plus engine startup such as CUDA graphs and warm-up.
- **What that means:** since new capacity arrives seconds late, **trigger early** and keep a **warm pool** ready. You can also shorten the load with faster storage or by loading fewer bytes, such as quantized weights.

Key numbers: cold start ≥ weight bytes ÷ load bandwidth.

Likely follow-up: "Which single metric would you pick?" `num_requests_waiting`, because it rises before latency does, which gives the autoscaler a head start.

> **TRAP**
>
> I first said GPU memory utilization, which never moves because of preallocation.

Build it yourself

- `exercises/06_08_09_tail_sim.py`: simulates how much a P99 estimate wobbles with sample size, and brute-forces the 20-call agent's 18%. Run `uv run exercises/06_08_09_tail_sim.py`.
- `exercises/24_27_find_walls.py`: finds the saturation, SLO and memory walls from a concurrency sweep and returns the binding capacity. Run `uv run exercises/24_27_find_walls.py`.